COMP 421 - Database Systems

Project 1: Database Design and Data Modelling

Group 99: Luca Mitran, Nicholas Imperioli, Ping-Chieh Tu

February 3rd 2025

Online Distribution Service of Computer Games Database Management

1 Requirement Analysis

1.1 Introduction

Today, computer games are not sold only as physical copies. Online distribution services provide a digital platform for purchasing, downloading, and managing games without having a physical copy. Most of them also offer cloud saving and multi-player connectivity. For developers, these platforms reduce their production cost, expand their audience, and also provide features such as community forums and regional pricing.

This ease of access is integral to developers as they have to spend less time worrying about these aspects of development and can put more effort into making an enjoyable gaming experience. For the scope of this project, we will focus on the interaction between the creation of a game and its journey to the hands of our users. This journey consists of features such as wishing for a game, discounts, and the purchasing process for buying said games.

We will act like a platform and focus our interest on the relations between the players and the games, between the developers and their games, and between fellow users.

1.2 Database Description

Entities and attributes:

- Developer: A developer is someone who creates games. A developer has a name, unique primary key did, and email.
- Publisher: A publisher is someone who publishes the game created by developers. A publisher has a unique primary key pid, name, and email.
- Game: A game must be made by a developer. Each game has a unique gid, rating, release date, and price.
- User: A user is someone who buys the games on our platform. Each user has a unique uid, email, and password.

- BillingInfo: This is a weak entity of a user. It has an address, name, and paymentMethod.
- Transaction: A transaction tracks purchases made by users This is also a weak entity of a user. It has a transid, gid, date, amount, and status.
- Discount: A game can have a discount/be on sale. It is a weak entity that has a startDate, endDate, and percentOff.
- Category: A category defines a game by genre and features. A game must have at least one category but can have multiple. It has a primary key name.

Relations:

- MakePublish: A game is made by a developer and published by a publisher. This is a ternary one-many-many relationship as a game can only be made and published by one developer and one publisher but developers and publishers can make and publish multiple games.
- Wish: A user wishes to own a game. This is a many-to-many relationship since a user can wish to have multiple games and a game can be wished by multiple users.
- InCart: A user adds a game to their cart but has yet to checkout. This is a many-to-many relationship since a user can add multiple games to their cart and a game can be added to cart by multiple users.
- Own: A game is owned by a user. This is a many-to-many relationship because a game can be owned by multiple users and a user can own multiple games.
- Buy: A user buys a game through a transaction. This is a ternary many-many-one relationship as a user can buy multiple games but a transaction can only be made by a user purchasing the items in their cart.
- Friend: A user can be friends with other users. This is a many-to-many relationship as a user can be friends of multiple users. It records the status between two users that have a connection.
- Belongs to: A game falls under a category. This is a many-to-many relationship as a game can fall under multiple categories and a category can have multiple games.
- Has: Encapsulate all weak entities with their parents. For example, a user has billing info.

1.3 Application Description

Our application is made to efficiently manage a digital game distribution platform with a focus on secure and optimal data management. To optimize our algorithm, we compile and process data during runtime. This includes user activity statistics, game metrics, and sales data; aggregating these statistics makes it easier to implement features like personalized recommendations as well as discount promotions at certain times. We dynamically perform the algorithms on our platform at scheduled intervals to avoid redundancy.

Preliminary calculations for the system are focused on user security and game management. This includes robust encryption for sensitive info like passwords and billing information and game metrics. We can also use these factors to change how games are presented in the store; by weighting

visibility based on rating, release date, and discount status, we can optimize game discovery and recommendations for users.

The main functions of our system include userLibrary(uid) which returns a user's owned games (with associated metadata) and calculateDiscount(gid, date) which determines the current price of a game after any promotions or discounts. The system also handles social features like friend relationships through a mutual confirmation system. We base our platform on dynamic processing of all these interconnected forms of data to make user experience more responsive, interactive, and manageable.

Several other variables managed by the system include: game pricing histories, user purchase patterns, and wishlist statistics. As already stated, securing sensitive data like payment information is very integral to the system. Furthermore, ensuring that there are mechanisms for maintaining accurate records of financial transactions in the system is an important feature for processing refunds and other user initiated requests.

1.4 Creative Description

Our online game distribution service database design demonstrates sophisticated modeling through several standout features. The system employs a ternary relationship (MakePublish) that realistically captures the complex interaction between games, developers, and publishers in the industry. The design also makes strategic use of weak entities for BillingInfo, Discount, and Transactions, which can elegantly handle time-sensitive and dependent data. The schema includes multiple many-to-many relationships that capture different states of user-game interactions, along with a self-referential relationship for the friend system. There is also a flexible categorization system, implemented through the BelongsTo relationship, that allows games to belong to multiple categories; this is flexible and scalable, allowing expansions in game discovery and organization capabilities. These features create a practical and sophisticated database design that reflects real industry practices while still being simple and expandable.

1.5 Inspirations

Web stores like Steam and Epic Games are two store fronts that inspired this database design.

1.6 Communication and Teamwork

The work was divvied up into equal parts based on the amount of marks for each section. One member worked on the introduction and database description, one worked on the e/r diagram, and one worked on the application and creative description of the database; However all work was collaborated on. Communication occurred through discord in regular intervals where we communicated necessary changes and feedback. After each part was done, we would ask for each other's input and what needed to be changed. If no changes were needed the work was pushed to the GitHub main branch.

2 Relation Tables

```
Developer(did,
          name NOT NULL,
          email NOT NULL,
          phone,
          PRIMARY KEY (did))
   Publisher(pid,
             name NOT NULL,
             email NOT NULL,
             phone,
             PRIMARY KEY (pid))
   Game(gid,
        date NOT NULL,
        rating,
        price NOT NULL,
        name NOT NULL,
        PRIMARY KEY (gid))
   Category(name,
            PRIMARY KEY (name))
   Users(uid,
         email NOT NULL,
         password NOT NULL,
         PRIMARY KEY (uid))
   BillingInfo(uid,
               address,
               name NOT NULL,
               payMethod NOT NULL,
               PRIMARY KEY (uid),
               FOREIGN KEY (uid) REFERENCES Users)
   Transactions(tid,
                date NOT NULL,
                amount NOT NULL,
                status NOT NULL,
                uid NOT NULL,
                PRIMARY KEY (tid),
                FOREIGN KEY (uid) REFERENCES Users)
   Discount(gid, startDate,
            endDate NOT NULL,
            percentOff NOT NULL,
            PRIMARY KEY (gid, startDate),
            FOREIGN KEY (gid) REFERENCES Game)
   MakePublish(gid, did NOT NULL, pid NOT NULL,
               PRIMARY KEY (gid),
               FOREIGN KEY (gid) REFERENCES Game,
               FOREIGN KEY (did) REFERENCES Developer,
               FOREIGN KEY (pid) REFERENCES Publisher)
```

```
Wish(uid, gid,
     PRIMARY KEY (uid, gid),
     FOREIGN KEY (uid) REFERENCES Users,
     FOREIGN KEY (gid) REFERENCES Game)
Own(uid, gid,
    PRIMARY KEY (uid, gid),
    FOREIGN KEY (uid) REFERENCES Users,
    FOREIGN KEY (gid) REFERENCES Game)
InCart(uid, gid,
       PRIMARY KEY (uid, gid),
       FOREIGN KEY (uid) REFERENCES Users,
       FOREIGN KEY (gid) REFERENCES Game)
Friend(uid1, uid2,
       status NOT NULL,
       PRIMARY KEY (uid1, uid2),
       FOREIGN KEY (uid1) REFERENCES Users,
       FOREIGN KEY (uid2) REFERENCES Users)
BelongsTo(gid, categoryName,
          PRIMARY KEY (gid, categoryName),
          FOREIGN KEY (gid) REFERENCES Game,
          FOREIGN KEY (categoryName) REFERENCES Category)
Buy(uid, gid, tid,
    PRIMARY KEY (uid, gid, tid),
    FOREIGN KEY (uid) REFERENCES Users,
    FOREIGN KEY (gid) REFERENCES Game,
    FOREIGN KEY (tid) REFERENCES Transactions)
```